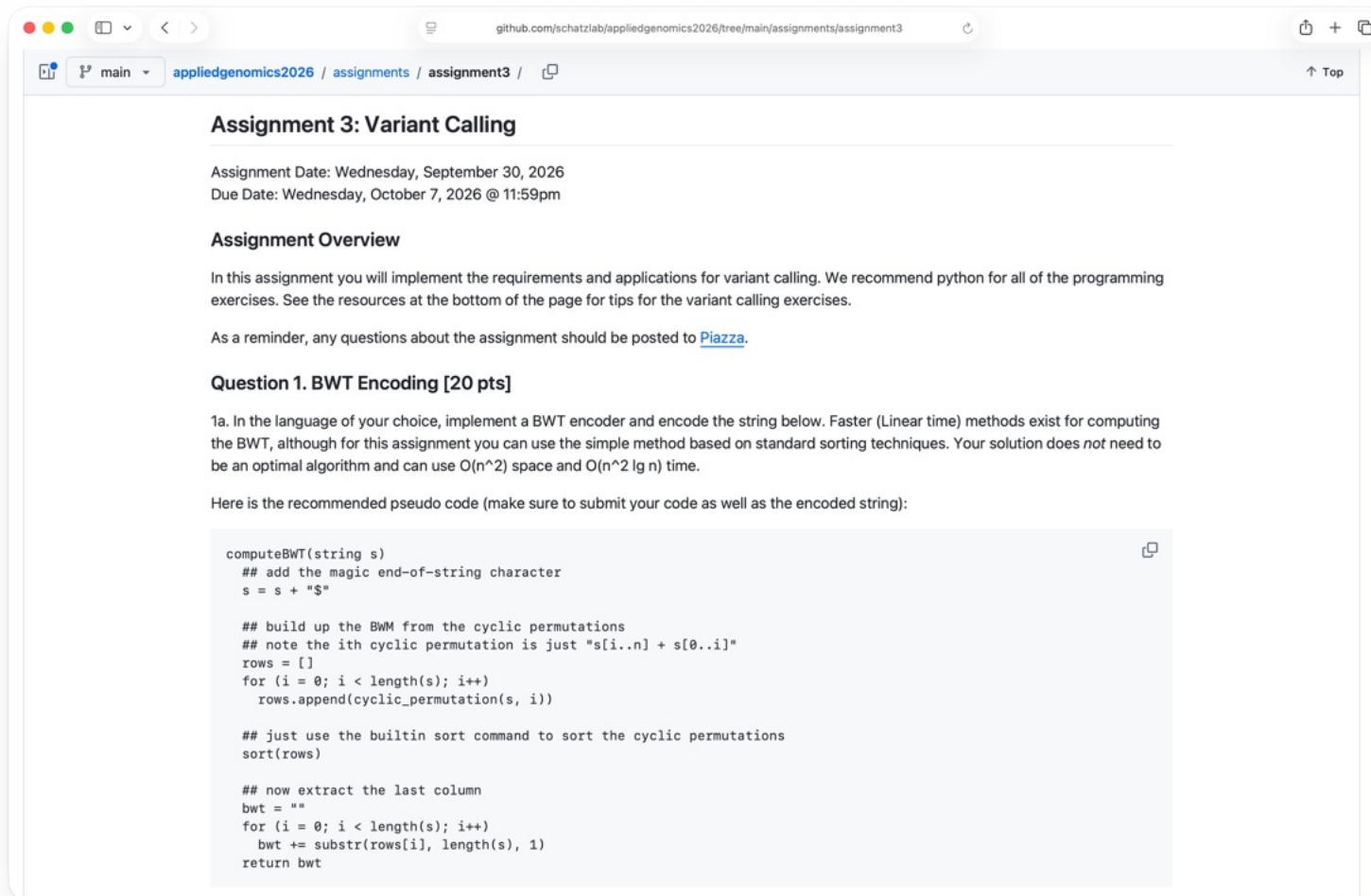


ImageNet Classification with Deep Convolutional Neural Networks

Michael Schatz
Oct 5, 2026

JHU EN.601.449/EN.601.649: Applied Comparative Genomics

Assignment 3 Due Wednesday Oct 7 by 11:59pm



The screenshot shows a web browser displaying a GitHub repository page. The browser's address bar shows the URL: `github.com/schatzlab/appliedgenomics2026/tree/main/assignments/assignment3`. The repository name is `appliedgenomics2026`, and the specific file or directory is `assignments/assignment3`. The page title is **Assignment 3: Variant Calling**. Below the title, the assignment date is listed as Wednesday, September 30, 2026, and the due date is Wednesday, October 7, 2026 @ 11:59pm. The section **Assignment Overview** explains that the assignment involves implementing requirements and applications for variant calling, recommending Python. It also mentions resources at the bottom of the page and a reminder to post questions to Piazza. The **Question 1. BWT Encoding [20 pts]** section contains a sub-question 1a asking for a BWT encoder implementation. It provides a recommended pseudo-code snippet for `computeBWT` in Python, which involves building up the BWT from cyclic permutations, sorting them, and extracting the last column.

Assignment 3: Variant Calling

Assignment Date: Wednesday, September 30, 2026
Due Date: Wednesday, October 7, 2026 @ 11:59pm

Assignment Overview

In this assignment you will implement the requirements and applications for variant calling. We recommend python for all of the programming exercises. See the resources at the bottom of the page for tips for the variant calling exercises.

As a reminder, any questions about the assignment should be posted to [Piazza](#).

Question 1. BWT Encoding [20 pts]

1a. In the language of your choice, implement a BWT encoder and encode the string below. Faster (Linear time) methods exist for computing the BWT, although for this assignment you can use the simple method based on standard sorting techniques. Your solution does *not* need to be an optimal algorithm and can use $O(n^2)$ space and $O(n^2 \lg n)$ time.

Here is the recommended pseudo code (make sure to submit your code as well as the encoded string):

```
computeBWT(string s)
    ## add the magic end-of-string character
    s = s + "$"

    ## build up the BWT from the cyclic permutations
    ## note the ith cyclic permutation is just "s[i..n] + s[0..i]"
    rows = []
    for (i = 0; i < length(s); i++)
        rows.append(cyclic_permutation(s, i))

    ## just use the builtin sort command to sort the cyclic permutations
    sort(rows)

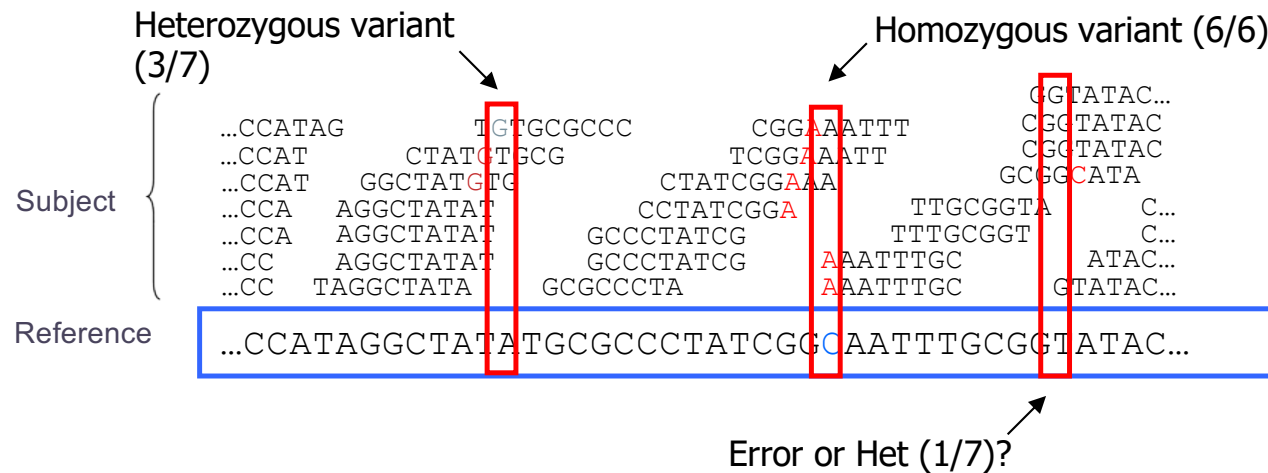
    ## now extract the last column
    bwt = ""
    for (i = 0; i < length(s); i++)
        bwt += substr(rows[i], length(s), 1)
    return bwt
```

Exam 1 Next Wednesday

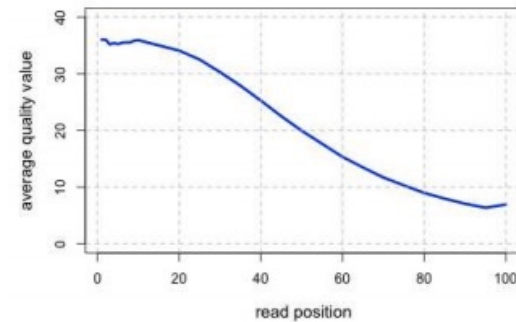
12	10/12/26	M	Review 1	Project Proposal	
13	10/14/26	W	Exam 1 (In class)		

Recap

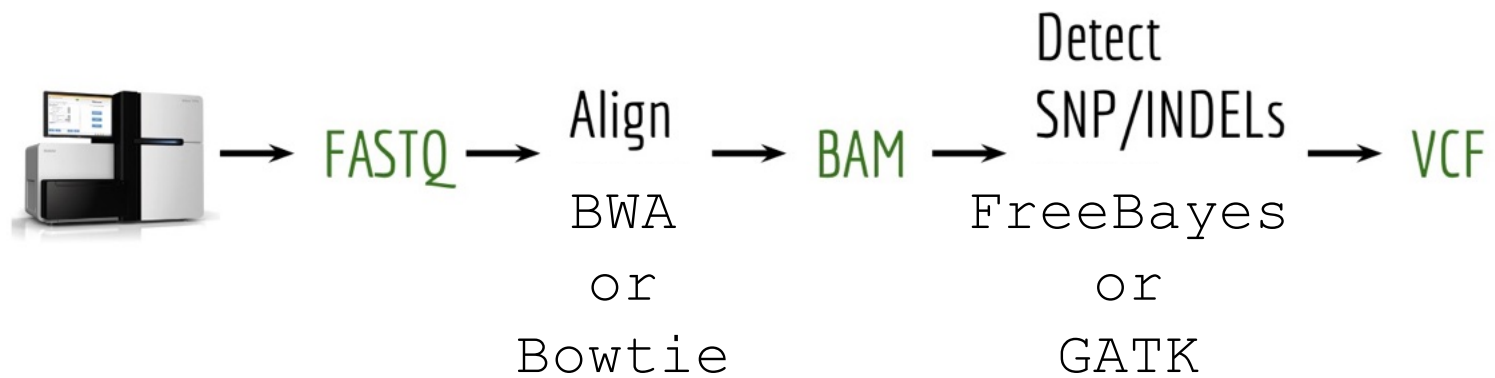
Genotyping Theory



- If there were no sequencing errors, identifying SNPs would be very easy: any time a read disagrees with the reference, it must be a variant!
- Sequencing instruments make mistakes
 - Quality of read decreases over the read length
- A single read differing from the reference is probably just an error, but it becomes more likely to be real as we see it multiple times



Variant Calling Overview



Algorithm Overview

1. Split read into segments

Read Read (reverse complement)

CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA TACAGGCCTGGGTAAAATAAGGCTGAGAGCTACTGG

Policy: extract 16 nt seed every 10 nt

Seeds

+, 0: CCAGTAGCTCTCAGCC +, 10: TCAGCCTTATTTTACC +, 20: TTTACCCAGGCCTGTA	-, 0: TACAGGCCTGGGTAAA -, 10: GGTAAAATAAGGCTGA -, 20: GGCTGAGAGCTACTGG
--	--

2. Lookup each segment and prioritize

Seeds

+, 0: CCAGTAGCTCTCAGCC

+, 10: TCAGCCTTATTTTACC

+, 20: TTTACCCAGGCCTGTA

-, 0: TACAGGCC TGGGTAAA

-, 10: GGTAAATAAGGCTGA

-, 20: GGCTGAGAGCTACTGG

Ungapped alignment with FM Index

aac

\$acaaag

aacg\$ac

acaaag\$

acg\$aca

Tcagccttat

g\$acaaac

Seed alignments (as B ranges)

{ [211, 212], [212, 214] }

{ [653, 654], [651, 653] }

{ [684, 685] }

{ }

{ }

{ [624, 625] }

3. Evaluate end-to-end match

Extension candidates

SA:684, chr12:1955
SA:624, chr2:462
SA:211, chr4:762
SA:213, chr12:1935
SA:652, chr12:1945

SIMD dynamic programming aligner

SAM alignments

r1 0 chr12 1936 0
36M * 0
CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA
IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
AS:i:0 XS:i:-2 XN:i:0
XM:i:0 XO:i:0 XG:i:0
NM:i:0 MD:Z:36 YT:Z:UU
YM:i:0

(Langmead & Salzberg, 2012)

Exact Matching Review & Overview

Where is GATTACA in the human genome?

Brute Force
(3 GB)

BANANA
BAN
ANA
NAN
ANA

$O(m * n)$

Slow & Easy

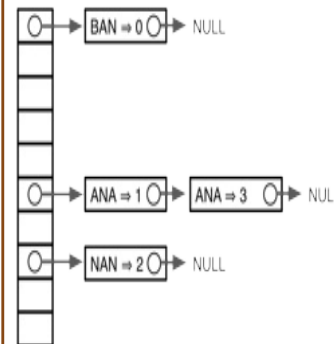
Suffix Array
(>15 GB)

6	\$
5	A\$
3	ANA\$
1	ANANA\$
0	BANANA\$
4	NA\$
2	NANA\$

$O(m + \lg n)$

Full-text index

Hash Table
(>15 GB)



$O(1)$

Fixed-length lookup

BWT
(3 GB)

BANANA\$
=>
\$BANANA
A\$BANANA
ANA\$BANANA
ANANA\$BANANA
BANANA\$BANANA
NA\$BANANA
NANA\$BANANA
=>
ANNB\$AA

$O(m)$

Full-text and concise

*** These are general techniques applicable to any text search problem ***

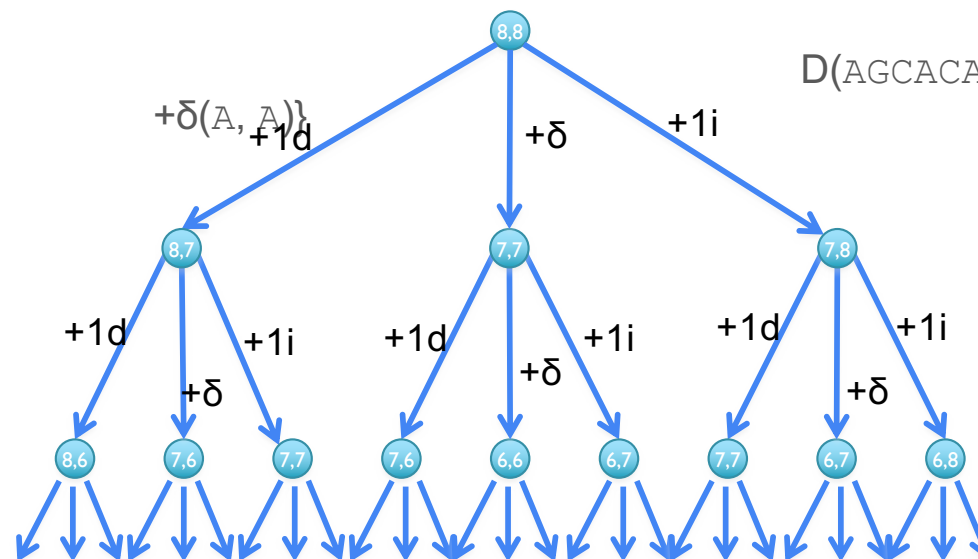
Recursive solution

- Computation of D is a recursive process.
 - At each step, we only allow matches, substitutions, and indels
 - $D(i,j)$ in terms of $D(i',j')$ for $i' \leq i$ and $j' \leq j$.

$$D(\text{AGCACACA}, \text{ACACACTA}) = \min\{D(\text{AGCACACA}, \text{ACACACT}) + 1,$$

$$D(\text{AGCACAC}, \text{ACACACTA}) + 1,$$

$$D(\text{AGCACAC}, \text{ACACACT})$$



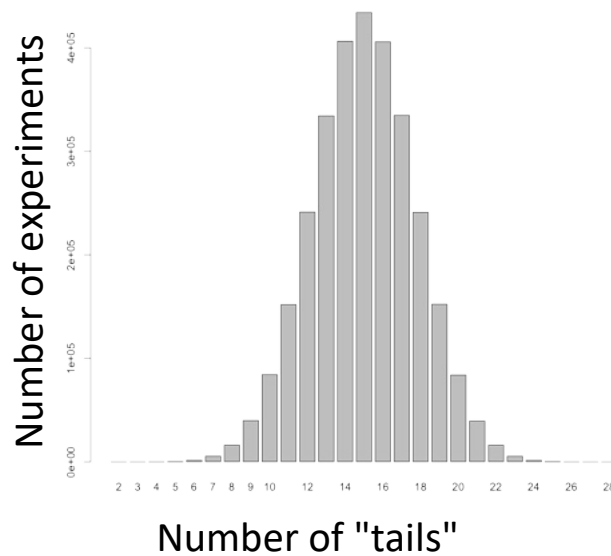
[What is the running time?]

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	1	2	3	4	5	6	7	8
A	1	0	1	2					
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A,ACA] = \min\{D[A,AC]+1, D[,ACA]+1, D[,AC]+\delta(A,A)\}$$

So, with 30 tosses (reads), we are much more likely to see an even mix of alternate and reference alleles at a heterozygous locus in a genome



This is why at least a "30X" (30 fold sequence coverage) genome is recommended: it confers sufficient power to distinguish heterozygous alleles and from mere sequencing errors

$$P(3/30 \text{ het}) <?> P(3/30 \text{ err})$$

PolyBayes: The first statistically rigorous variant detection tool.

letter

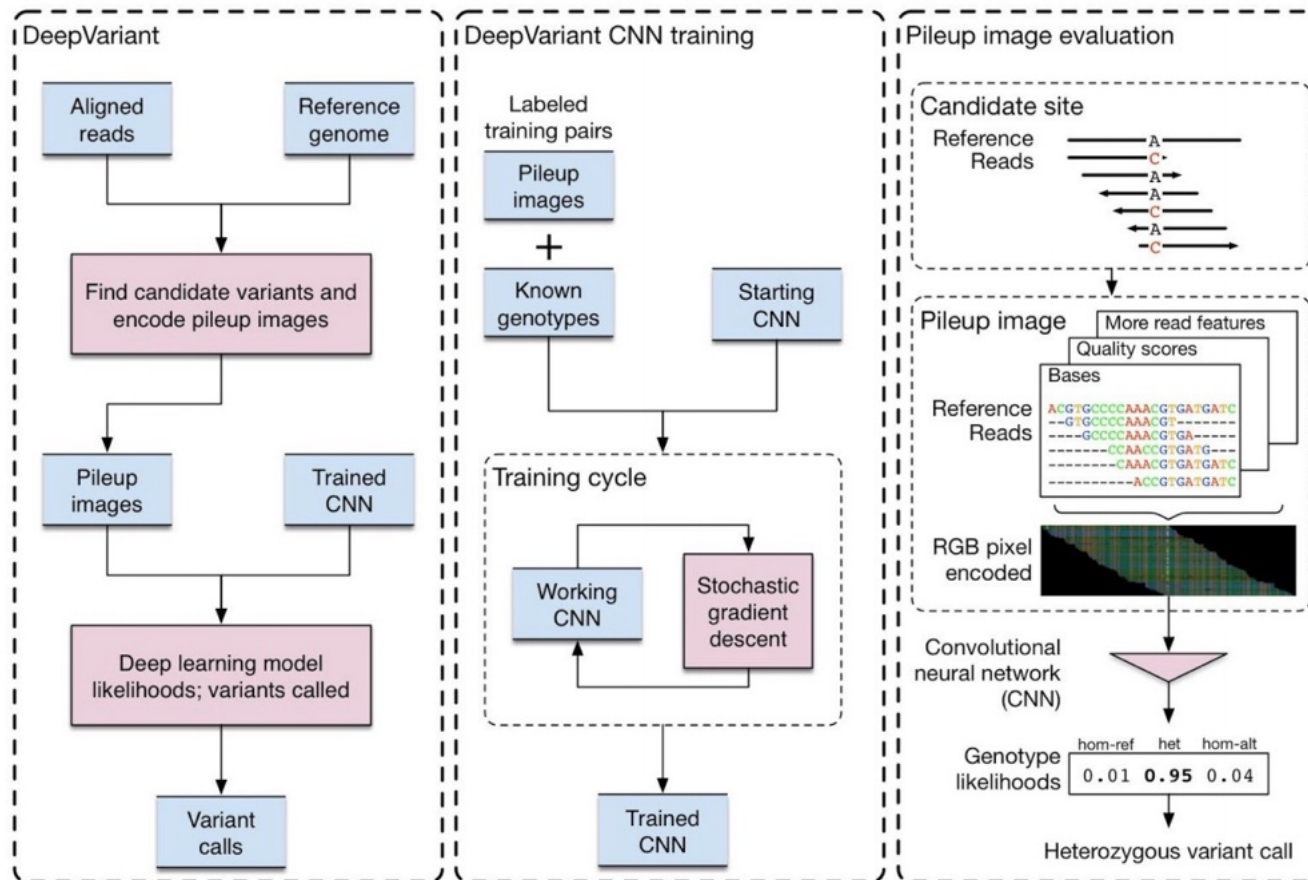
© 1999 Nature America Inc. • <http://genetics.nature.com>

A general approach to single-nucleotide polymorphism discovery

Gabor T. Marth¹, Ian Korf¹, Mark D. Yandell¹, Raymond T. Yeh¹, Zhijie Gu², Hamideh Zakeri²,
Nathan O. Stitzel¹, LaDeana Hillier¹, Pui-Yan Kwok² & Warren R. Gish¹

This Bayesian statistical framework
has been adopted by most other
modern SNP/INDEL callers such as
FreeBayes, GATK, and samtools

DeepVariant



Creating a universal SNP and small indel variant caller with deep neural networks

Poplin et al. (2018) Nature Biotechnology. <https://www.nature.com/articles/nbt.4235>

AlexNet

“AlexNet”

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called “dropout” that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.



Alex Krizhevsky
U. Toronto/Google



Ilya Sutskever
U. Toronto/OpenAI/
Safe Superintelligence Inc



Geoffrey Hinton
U. Toronto/Vector Institute

“AlexNet”

Imagenet classification with deep convolutional neural networks

Authors Alex Krizhevsky, Ilya Sutskever, Geoffrey E Hinton

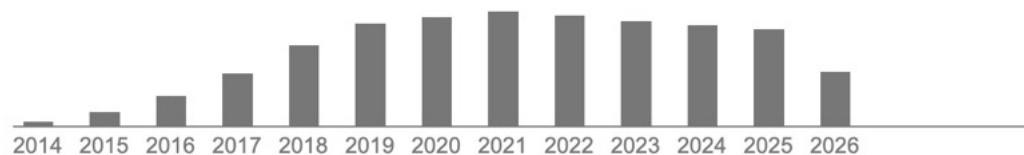
Publication date 2012

Journal Advances in neural information processing systems

Volume 25

Description We trained a large, deep convolutional neural network to classify the 1.3 million high-resolution images in the LSVRC-2010 ImageNet training set into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 39.7% and 18.9% which is considerably better than the previous state-of-the-art results. The neural network, which has 60 million parameters and 500,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and two globally connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of convolutional nets. To reduce overfitting in the globally connected layers we employed a new regularization method that proved to be very effective.

Total citations [Cited by 200830](#)



“AlexNet”

Imagenet classification with deep convolutional neural networks

Authors Alex Krizhevsky, Ilya Sutskever, Geoffrey E Hinton

Publication date 2012

Journal Advances in neural information processing systems

Volume 25

Description We trained a large, deep convolutional neural network to classify the 1.3 million high-resolution images in the LSVRC-2010 ImageNet training set into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 39.7% and 18.9% which is considerably better than the previous state-of-the-art results. The neural network, which has 60 million parameters and 500,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and two globally connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of convolutional nets. To reduce overfitting in the globally connected layers we employed a new regularization method that proved to be very effective.

Total citations Cited by 200830



Outline

1. The Problem
2. pre-AlexNet approaches
3. How do animals & humans approach this?
4. Artificial Neural Networks
5. Convolutional Neural Networks
6. AlexNet
7. Impact

Outline

1. The Problem
2. pre-AlexNet approaches
3. How do animals & humans approach this?
4. Artificial Neural Networks
5. Convolutional Neural Networks
6. AlexNet
7. Impact

What is this?



What is this?



Fish



Axe



Bagel



Dog



Van

ImageNet Classification Challenge

SUN, 131K

[Xiao et al. '10]

LabelMe, 37K

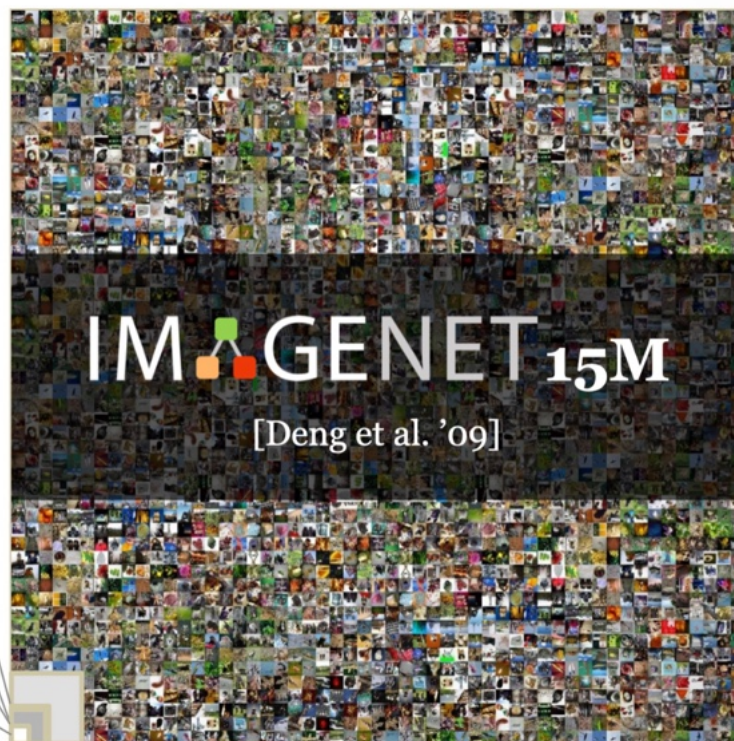
[Russell et al. '07]

PASCAL VOC, 30K

[Everingham et al. '06-'12]

Caltech101, 9K

[Fei-Fei, Fergus, Perona, '03]



- 15M images organized into 21,841 categories
- ILSVRC (AlexNet) uses a subset:
 - 1000 images in 1000 categories
 - 1.2M training, 50k validation, 150k test

imagenet_ilsvrc2017_v1.0

ImageNet Examples

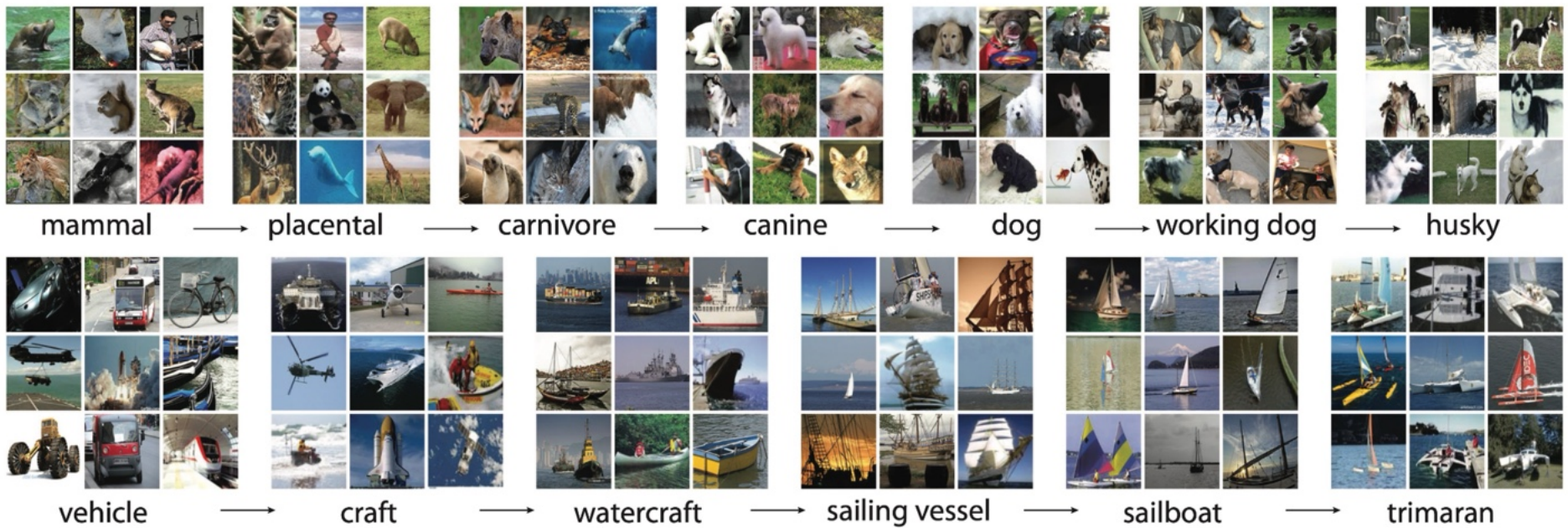


Figure 1: A snapshot of two root-to-leaf branches of ImageNet: the **top** row is from the mammal subtree; the **bottom** row is from the vehicle subtree. For each synset, 9 randomly sampled images are presented.

Images organized into nested categories determined by “WordNet”

Classification Challenges



husky

- “Natural” variation between individuals
 - Different colors, shapes, sizes
 - Very similar categories: 90 different dog breeds in the challenge (120 total)
- “Technical” variations
 - Orientations: side, front, behind, etc
 - Distance & position of the object
 - Other objects in the image

Classification Challenges



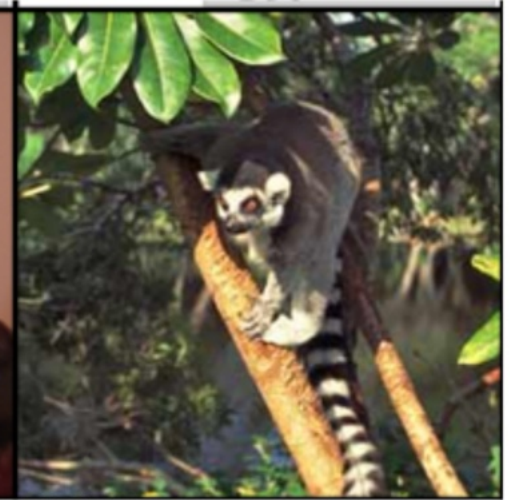
grille



mushroom



cherry



Madagascar cat

convertible	agaric	dalmatian	squirrel monkey
grille	mushroom	grape	spider monkey
pickup	jelly fungus	elderberry	titi
beach wagon	gill fungus	ffordshire bullterrier	indri
fire engine	dead-man's-fingers	currant	howler monkey

Classification “errors” from AlexNet

Outline

1. The Problem
2. pre-AlexNet approaches
3. How do animals & humans approach this?
4. Artificial Neural Networks
5. Convolutional Neural Networks
6. AlexNet
7. Impact

High-Dimensional Signature Compression for Large-Scale Image Classification

Jorge Sánchez and Florent Perronnin
Textual and Visual Pattern Analysis (TVPA) group
Xerox Research Centre Europe (XRCE)

Abstract

We address image classification on a large-scale, i.e. when a large number of images and classes are involved. First, we study classification accuracy as a function of the image signature dimensionality and the training set size. We show experimentally that the larger the training set, the higher the impact of the dimensionality on the accuracy. In other words, high-dimensional signatures are important to obtain state-of-the-art results on large datasets. Second, we tackle the problem of data compression on very large signatures (on the order of 10^5 dimensions) using two lossy compression strategies: a dimensionality reduction technique known as the hash kernel and an encoding technique based on product quantizers. We explain how the gain in storage can be traded against a loss in accuracy and/or an increase in CPU cost. We report results on two large databases – ImageNet and a dataset of 1M Flickr images – showing that we can reduce the storage of our signatures by a factor 64 to 128 with little loss in accuracy. Integrating the decompression in the classifier learning yields an efficient and scalable training algorithm. On ILSVRC2010 we report a 74.3% accuracy at top-5, which corresponds to a 2.5% absolute improvement with respect to the state-of-the-art. On a subset of 10K classes of ImageNet we report a top-1 accuracy of 16.7%, a relative improvement of 160% with respect to the state-of-the-art.

1. Introduction

Scaling-up image classification systems is a problem which is receiving an increasing attention as larger labeled image datasets are becoming available. For instance, ImageNet (www.image-net.org) consists of more than 12M images of 17K concepts [7] and Flickr contains thousands of groups (www.flickr.com/groups) – some of which with hundreds of thousands of pictures – which can be readily used to learn object classifiers [31, 22].

The focus in the image classification community was initially on developing systems which would yield the best possible accuracy fairly independently of their cost. The

winners of the PASCAL VOC 2007 [8] and 2008 [9] competitions used a similar paradigm: many types of low-level local features are extracted (referred to as “channels”), one bag-of-visual-words (BOV) histogram is computed for each channel and non-linear kernel classifiers such as SVMs are used to perform classification [38, 29]. The use of many channels and costly non-linear SVMs was made possible by the modest size of the available databases.

In recent years only has the computational cost become a central issue in image classification / object detection. In [19], Maji *et al.* showed that the runtime cost of an intersection kernel (IK) SVM could be made independent of the number of support vectors. Maji and Berg [18] and Wang *et al.* [31] then proposed efficient algorithms to learn IK-SVMs. Vedaldi and Zisserman [30] and Perronnin *et al.* [21] subsequently generalized this principle to any additive classifier. Another line of research consists in computing image representations which are directly amenable to cost-less linear classification. Yang *et al.* [36], Wang *et al.* [32] and Boureau *et al.* [4] showed that replacing the average pooling stage in the BOV computation by a max-pooling yielded excellent results. To go beyond the BOV, i.e. beyond counting, it has been proposed to include higher order statistics in the image signature. This includes modeling an image by a probability distribution [17, 35] or using the Fisher kernel framework [20]. Especially, it was shown that the Fisher Vector (FV) could yield high accuracy with linear classifiers [22].

If one wants to stick to efficient linear classifiers, the image representations should be high-dimensional to ensure linear separability of the classes. Therefore, we argue that the storage/memory cost is becoming a central issue in large-scale image classification. As an example, in this paper we consider almost dense image representations – based on the improved FV of [22] – with up to 524K dimensions. Using a 4 byte floating point representation, a single signature requires 2MB of storage. Storing the ILSVRC2010 dataset [2] would take approximately 2.8TBs and storing the full ImageNet dataset around 23TBs. Obviously, these numbers have to be multiplied by the number of channels, i.e. feature types. As another example, the

1. Break an image into (many) patches
2. Transform local features into “visual words” using a “code book”
3. Use the visual words to create a sparse feature vector
4. Compare the vector to the vectors precomputed from your database

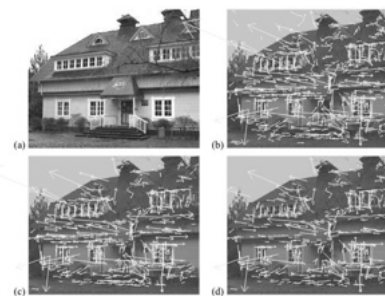
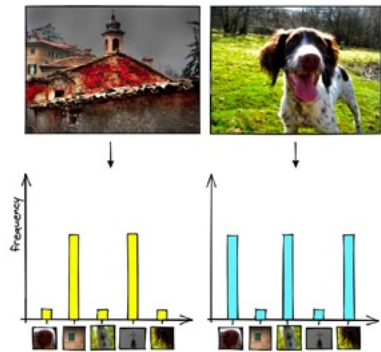


Figure 5: This figure shows the stages of keypoint selection. (a) The 233x189 pixel original image. (b) The initial 832 keypoints locations at maxima and minima of the difference-of-Gaussian function. Keypoints are displayed as vectors indicating scale, orientation, and location. (c) After applying a threshold on minimum contrast, 729 keypoints remain. (d) The final 536 keypoints that remain following an additional threshold on ratio of principal curvatures.



“SIFT descriptors”

“Visual word histogram”

The winner in 2011

High-Dimensional Signature Compression for Large-Scale Image Classification

Jorge Sánchez and Florent Perronnin
Textual and Visual Pattern Analysis (TVPA) group
Xerox Research Centre Europe (XRCE)

Abstract

We address image classification on a large-scale, i.e. when a large number of images and classes are involved. First, we study classification accuracy as a function of the image signature dimensionality and the training set size. We show experimentally that the larger the training set, the higher the impact of the dimensionality on the accuracy. In other words, high-dimensional signatures are important to obtain state-of-the-art results on large datasets. Second, we tackle the problem of data compression on very large signatures (on the order of 10^5 dimensions) using two lossy compression strategies: a dimensionality reduction technique known as the hash kernel and an encoding technique based on product quantizers. We explain how the gain in storage can be traded against a loss in accuracy and/or an increase in CPU cost. We report results on two large databases – ImageNet and a dataset of 1M Flickr images – showing that we can reduce the storage of our signatures by a factor 64 to 128 with little loss in accuracy. Integrating the decomposition in the classifier learning yields an efficient and scalable training algorithm. On ILSVRC2010 we report a 74.3% accuracy at top-5, which corresponds to a 2.5% absolute improvement with respect to the state-of-the-art. On a subset of 10K classes of ImageNet we report a top-1 accuracy of 16.7%, a relative improvement of 160% with respect to the state-of-the-art.

1. Introduction

Scaling-up image classification systems is a problem which is receiving an increasing attention as larger labeled image datasets are becoming available. For instance, ImageNet (www.image-net.org) consists of more than 12M images of 17K concepts [7] and Flickr contains thousands of groups (www.flickr.com/groups) – some of which with hundreds of thousands of pictures – which can be readily used to learn object classifiers [31, 22].

The focus in the image classification community was initially on developing systems which would yield the best possible accuracy fairly independently of their cost. The

winners of the PASCAL VOC 2007 [8] and 2008 [9] competitions used a similar paradigm: many types of low-level local features are extracted (referred to as “channels”), one bag-of-visual-words (BOV) histogram is computed for each channel and non-linear kernel classifiers such as SVMs are used to perform classification [38, 29]. The use of many channels and costly non-linear SVMs was made possible by the modest size of the available databases.

In recent years only has the computational cost become a central issue in image classification / object detection. In [19], Maji *et al.* showed that the runtime cost of an intersection kernel (IK) SVM could be made independent of the number of support vectors. Maji and Berg [18] and Wang *et al.* [31] then proposed efficient algorithms to learn IKSVMs. Vedaldi and Zisserman [30] and Perronnin *et al.* [21] subsequently generalized this principle to any additive classifier. Another line of research consists in computing image representations which are directly amenable to cost-less linear classification. Yang *et al.* [36], Wang *et al.* [32] and Boureau *et al.* [4] showed that replacing the average pooling stage in the BOV computation by a max-pooling yielded excellent results. To go beyond the BOV, i.e. beyond counting, it has been proposed to include higher order statistics in the image signature. This includes modeling an image by a probability distribution [17, 35] or using the Fisher kernel framework [20]. Especially, it was shown that the Fisher Vector (FV) could yield high accuracy with linear classifiers [22].

If one wants to stick to efficient linear classifiers, the image representations should be high-dimensional to ensure linear separability of the classes. Therefore, we argue that *the storage/memory cost is becoming a central issue in large-scale image classification*. As an example, in this paper we consider almost dense image representations – based on the improved FV of [22] – with up to 524K dimensions. Using a 4 byte floating point representation, a single signature requires 2MB of storage. Storing the ILSVRC2010 dataset [2] would take approximately 2.8TBs and storing the full ImageNet dataset around 23TBs. Obviously, these numbers have to be multiplied by the number of channels, i.e. feature types. As another example, the

Limitations

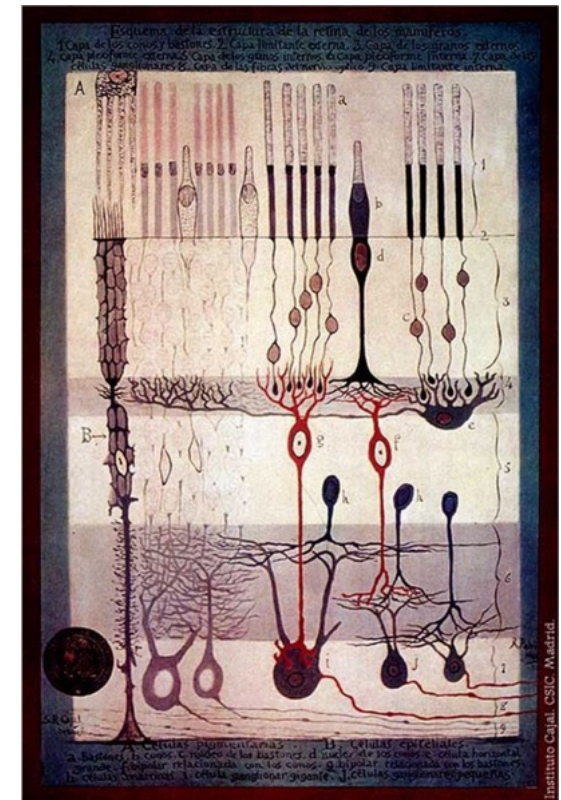
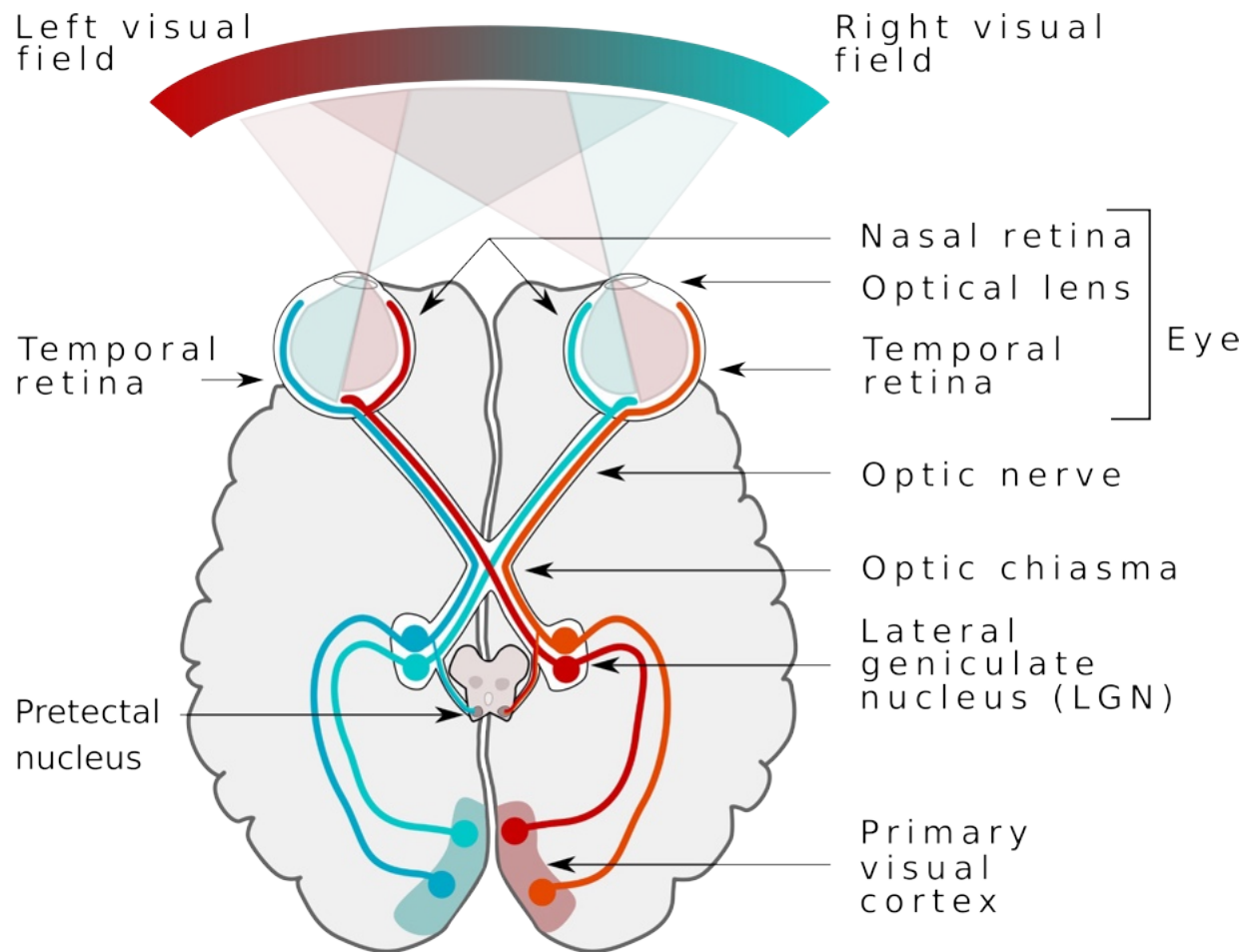
- Ignores spatial relationships between patches (“a bag of words”)
- High dependency of local features that are (often) hand crafted to maximize performance on the training data
- Computationally demanding to extract features, perform high dimensional comparisons
- >25% error

The winner in 2011

Outline

1. The Problem
2. pre-AlexNet approaches
3. How do animals & humans approach this?
4. Artificial Neural Networks
5. Convolutional Neural Networks
6. AlexNet
7. Impact

Human Visual Cortex



Structure of the Mammalian Retina
S. Ramón y Cajal (1900)

Vision & Pattern Recognition

Receptive Fields, Binocular Interaction and Functional Architecture in the Cat's Visual Cortex

D. H. HUBEL AND T. N. WIESEL • *Neurophysiology Laboratory, Department of Pharmacology Harvard Medical School, Boston, Massachusetts*

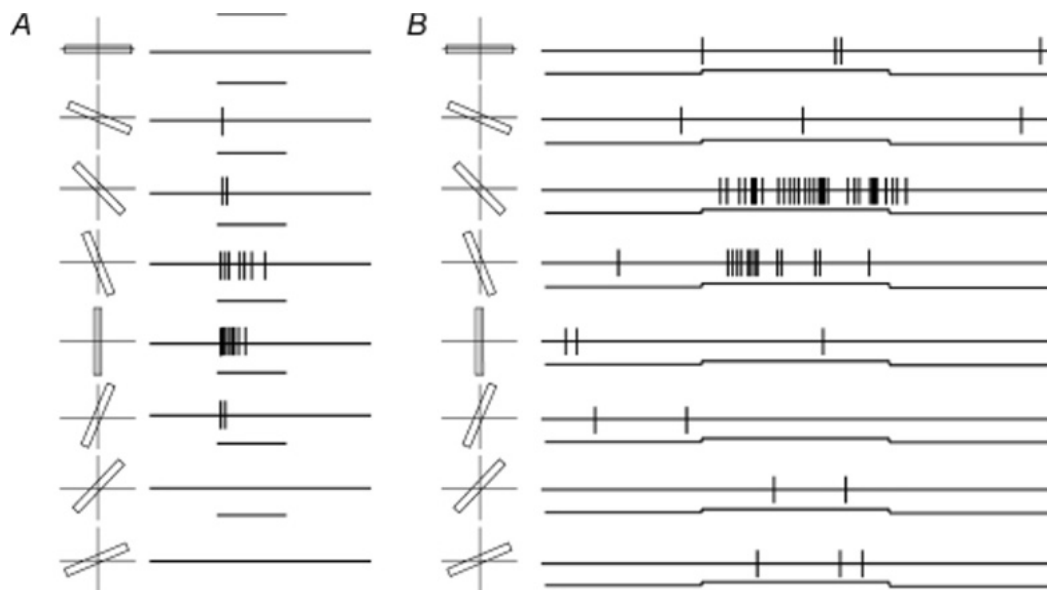


Table 1. Simple Cortical Fields

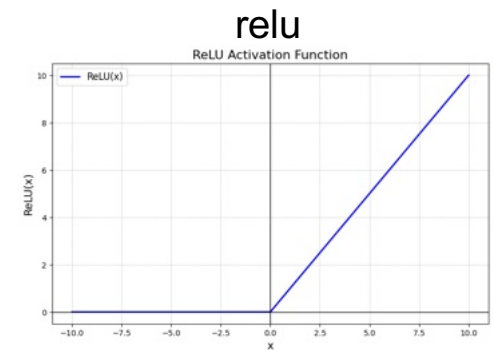
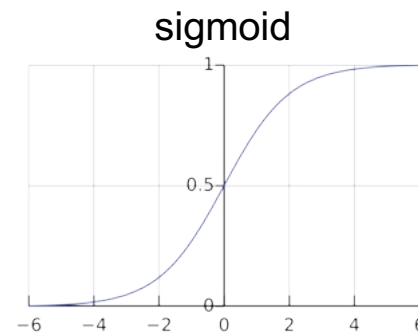
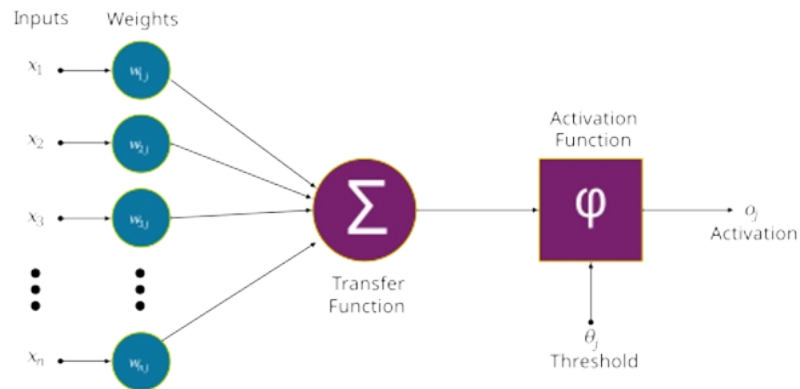
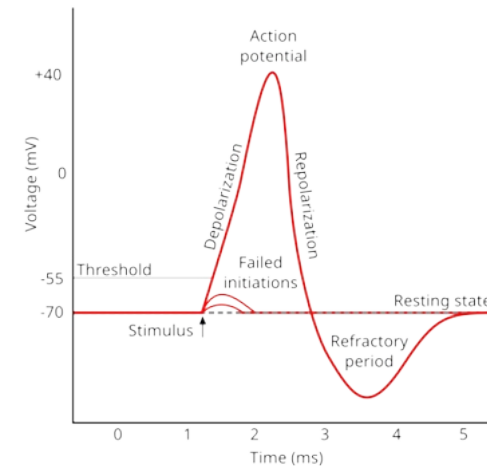
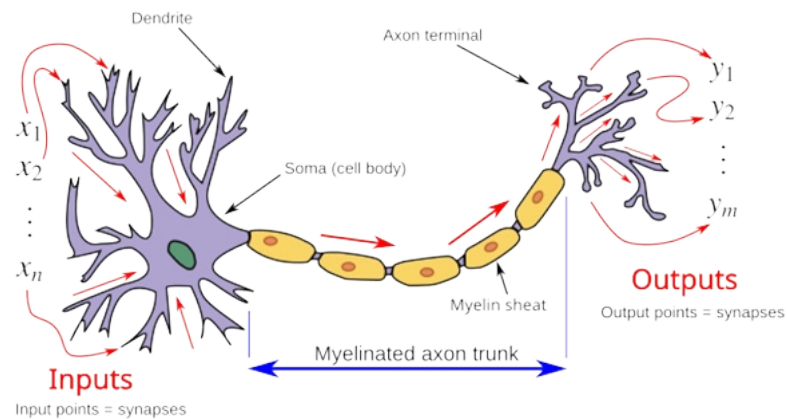
	<i>Text-fig.</i>	<i>No. of cells</i>
(a) Narrow concentrated centres		
(i) Symmetrical flanks		
Excitatory centres	2C	23
Inhibitory centres	2D	17
(ii) Asymmetrical flanks		
Excitatory centres	—	28
Inhibitory centres	2E	10
(b) Large centres; concentrated flanks	2F	21
(c) One excitatory region and one inhibitory	2G	17
(d) Uncategorized	—	117
Total number of simple fields		233

Example simple cells in the striate cortex of anaesthetized paralysed cat (A) and the awake fixating monkey (B). The comparison illustrates the similar response to oriented slits of light in the anaesthetized, paralysed cats and awake behaving monkeys. The example from the awake monkey shows the same qualitative orientation tuning but a slightly higher background rate than that from the anaesthetized cat. Traced from Fig. 3A of Hubel & Wiesel (1959) and from Fig. 5 of (Wurtz, 1969c).

Outline

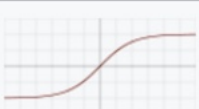
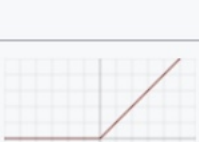
1. The Problem
2. pre-AlexNet approaches
3. How do animals & humans approach this?
4. Artificial Neural Networks
5. Convolutional Neural Networks
6. AlexNet
7. Impact

Biological vs Artificial Neurons



The non-linear activation function (sigmoid/relu) is needed to model non-linear relationships

ReLU Activation

Name	Plot	Function, $g(x)$	Derivative of g , $g'(x)$	Range	Order of continuity
Identity		x	1	$(-\infty, \infty)$	C^∞
Binary step		$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	0	$\{0, 1\}$	C^{-1}
Logistic, sigmoid, or soft step		$\sigma(x) \doteq \frac{1}{1 + e^{-x}}$	$g(x)(1 - g(x))$	$(0, 1)$	C^∞
Hyperbolic tangent ($\tanh$)		$\tanh(x) \doteq \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$1 - g(x)^2$	$(-1, 1)$	C^∞
Soboleva modified hyperbolic tangent (smht)		$\text{smht}(x) \doteq \frac{e^{ax} - e^{-bx}}{e^{cx} + e^{-dx}}$		$(-1, 1)$	C^∞
Rectified linear unit (ReLU) ^[10]		$(x)^+ \doteq \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$ $= \max(0, x) = x \mathbf{1}_{x>0}$	$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$	$[0, \infty)$	C^0

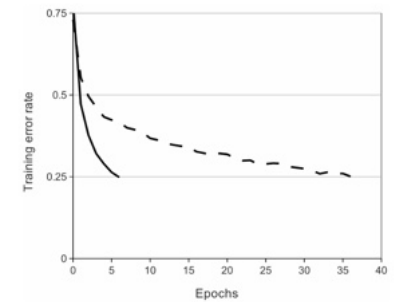
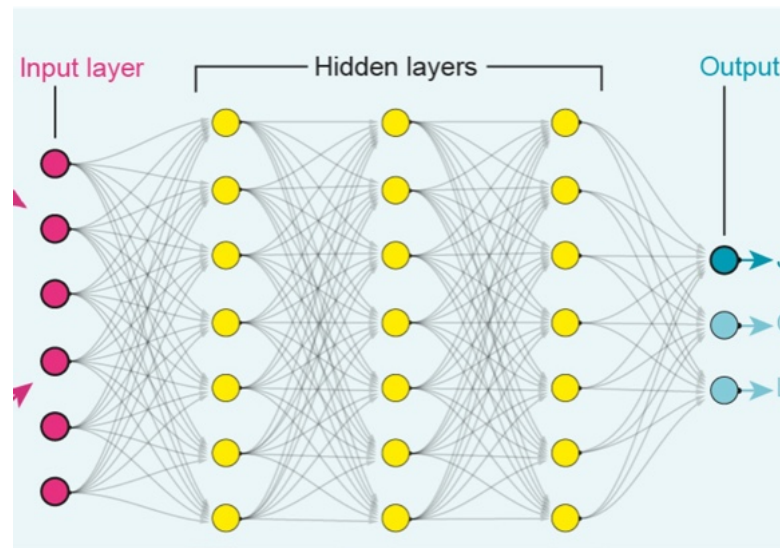


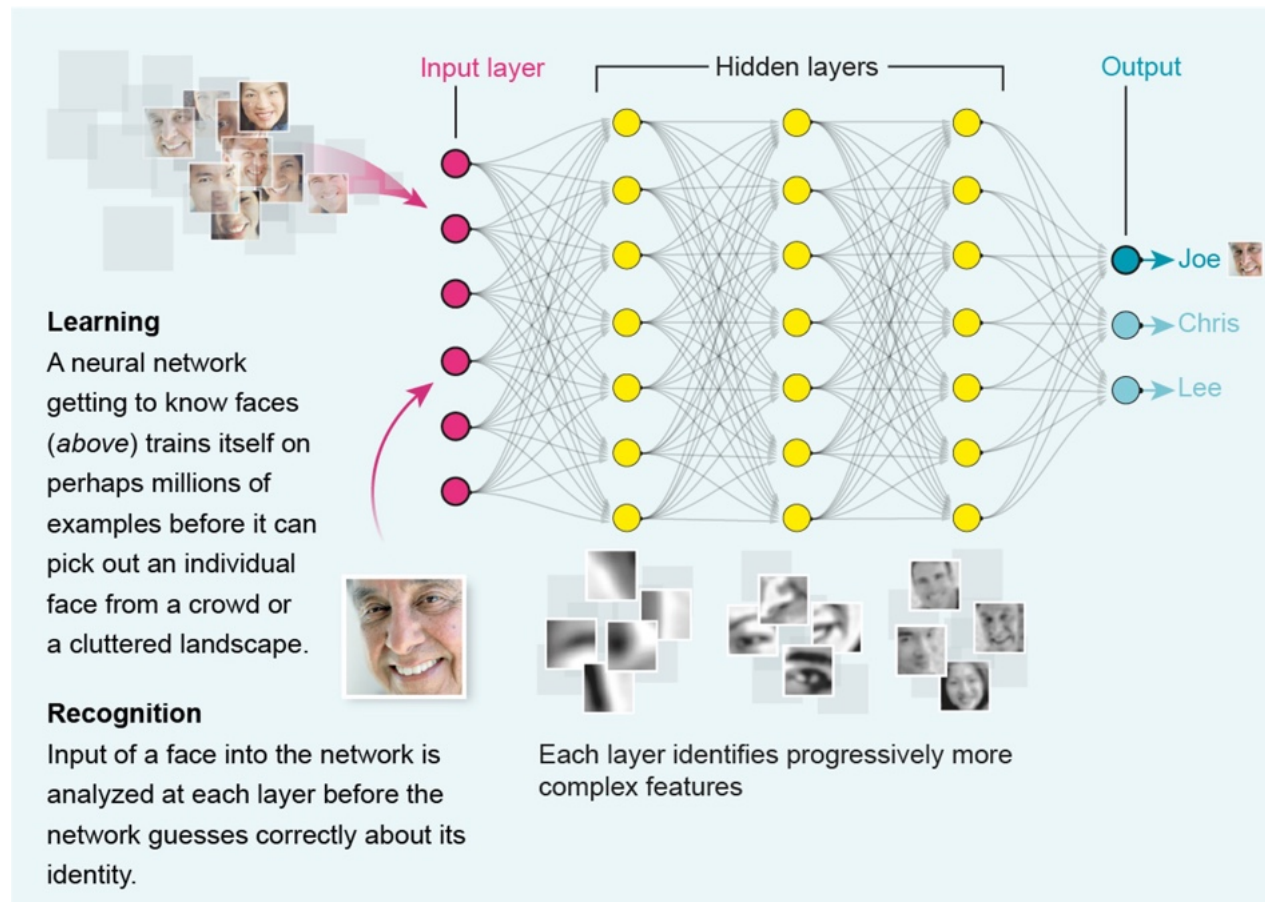
Figure 1: A four-layer convolutional neural network with ReLUs (**solid line**) reaches a 25% training error rate on CIFAR-10 six times faster than an equivalent network with tanh neurons (**dashed line**). The learning rates for each network were chosen independently to make training as fast as possible. No regularization of any kind was employed. The magnitude of the effect demonstrated here varies with network architecture, but networks with ReLUs consistently learn several times faster than equivalents with saturating neurons.

Artificial Neural Networks



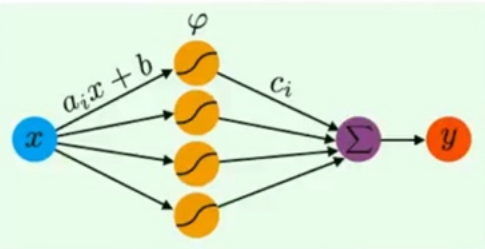
<https://www.codingdojo.com/blog/artificial-neural-network-brain-behind-todays-smart-technology>

Artificial Neural Networks



<https://www.codingdojo.com/blog/artificial-neural-network-brain-behind-todays-smart-technology>

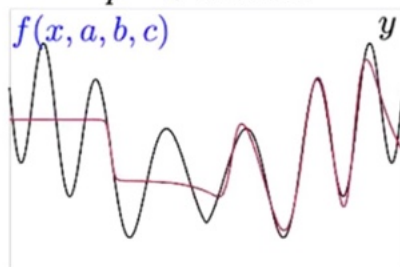
ANNs are “Universal Approximators”



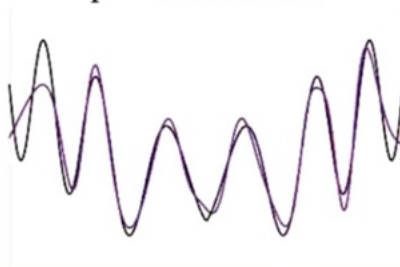
1 hidden layer perceptron:

$$y \approx f(x, a, b, c) \stackrel{\text{def.}}{=} \sum_{i=1}^p c_i \varphi(a_i x + b_i)$$

$p = 6$ neurons



$p = 20$ neurons



Approximation by Superpositions of a Sigmoidal Function*

G. Cybenko†

Abstract. In this paper we demonstrate that finite linear combinations of compositions of a fixed, univariate function and a set of affine functionals can uniformly approximate any continuous function of n real variables with support in the unit hypercube; only mild conditions are imposed on the univariate function. Our results settle an open question about representability in the class of single hidden layer neural networks. In particular, we show that arbitrary decision regions can be arbitrarily well approximated by continuous feedforward neural networks with only a single internal, hidden layer and any continuous sigmoidal nonlinearity. The paper discusses approximation properties of other possible types of nonlinearities that might be implemented by artificial neural networks.

Key words. Neural networks, Approximation, Completeness.



George Cybenko

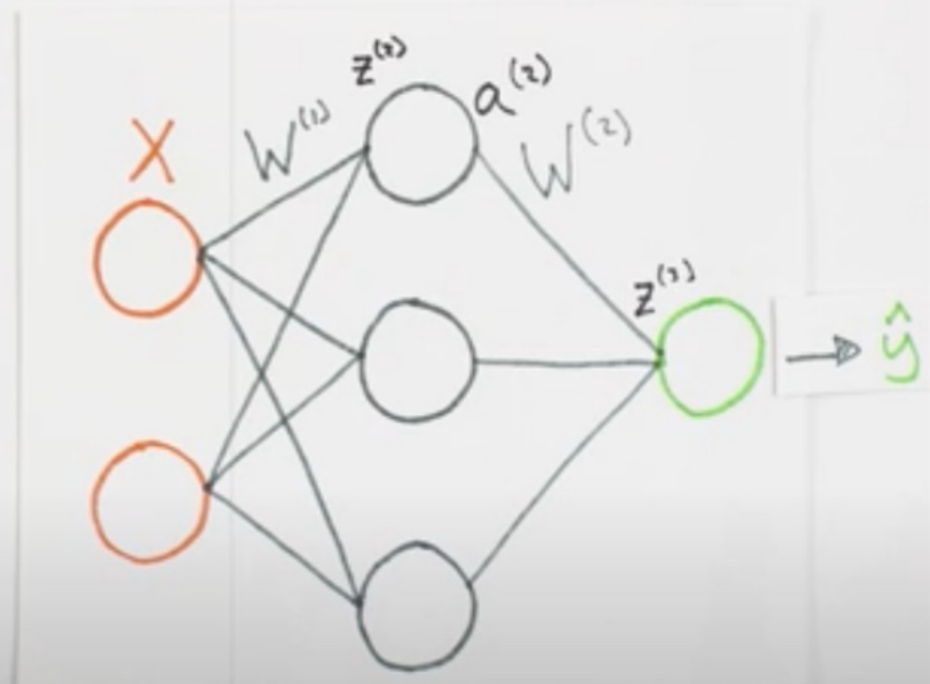
Training with Backpropagation: Random initialization

```
class Neural_Network(object):
    def __init__(self):
        #Define Hyperparameters
        self.inputLayerSize = 2
        self.outputLayerSize = 1
        self.hiddenLayerSize = 3

        #Weights (Parameters)
        self.W1 = np.random.randn(self.inputLayerSize, \
                                    self.hiddenLayerSize)
        self.W2 = np.random.randn(self.hiddenLayerSize, \
                                    self.outputLayerSize)

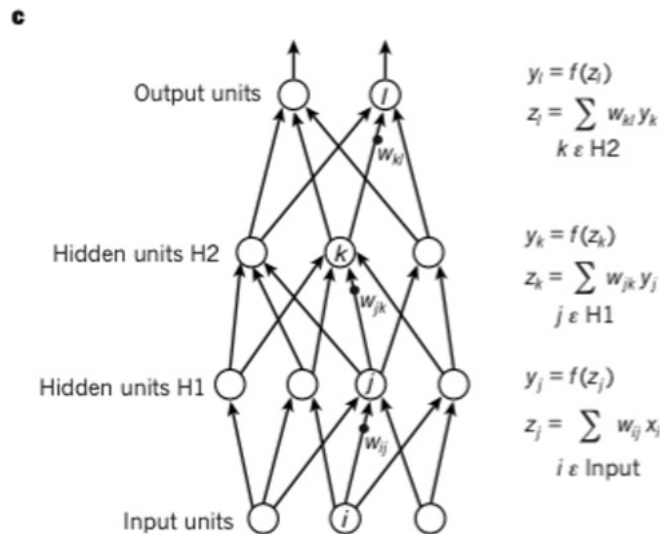
    def forward(self, X):
        #Propagate inputs though network
        self.z2 = np.dot(X, self.W1)
        self.a2 = self.sigmoid(self.z2)
        self.z3 = np.dot(self.a2, self.W2)
        yHat = self.sigmoid(self.z3)
        return yHat

    def sigmoid(self, z):
        #Apply sigmoid activation function to scalar, vector, or matrix
        return 1/(1+np.exp(-z))
```

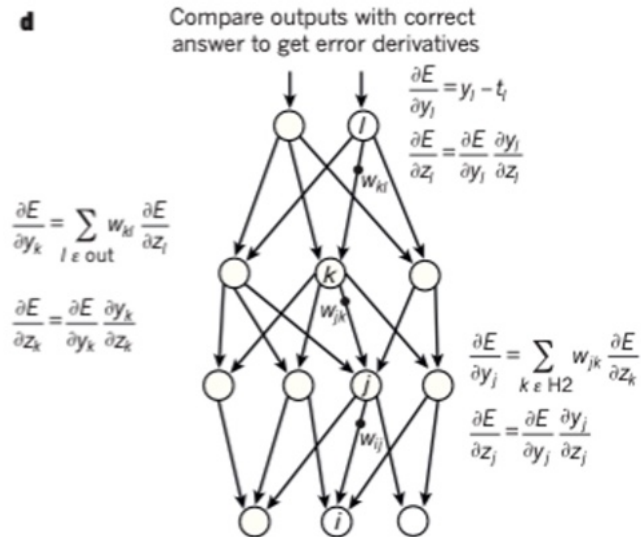


Training with Backpropagation

Forward pass: evaluation



Reverse pass: backpropagation



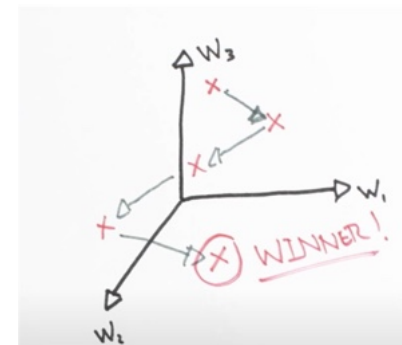
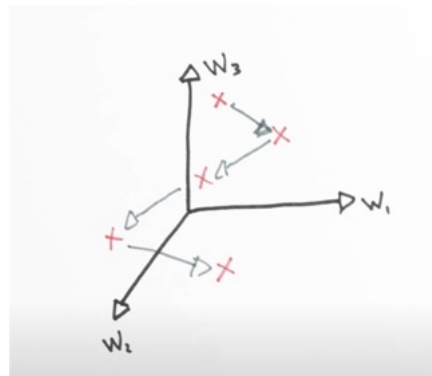
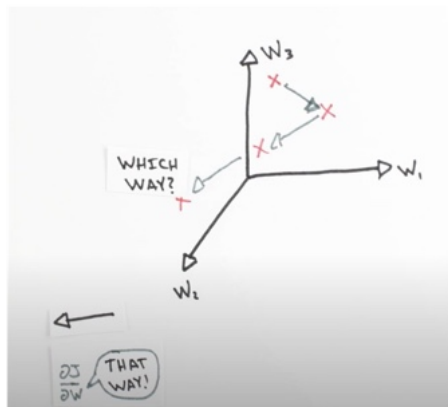
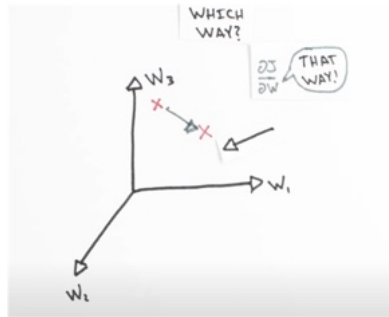
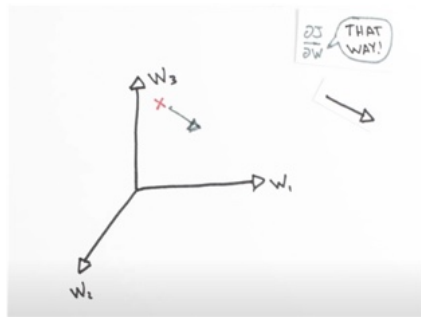
- Calculations along every edge, but the math is generally very basic
 - Addition, multiplication, exponentiation, etc
- Also easy to parallelize (especially with GPUs)

Deep Learning

LeCun, Bengio, Hinton (2020). Nature. doi:10.1038/nature14539

Training with Backpropagation: Gradient Descent

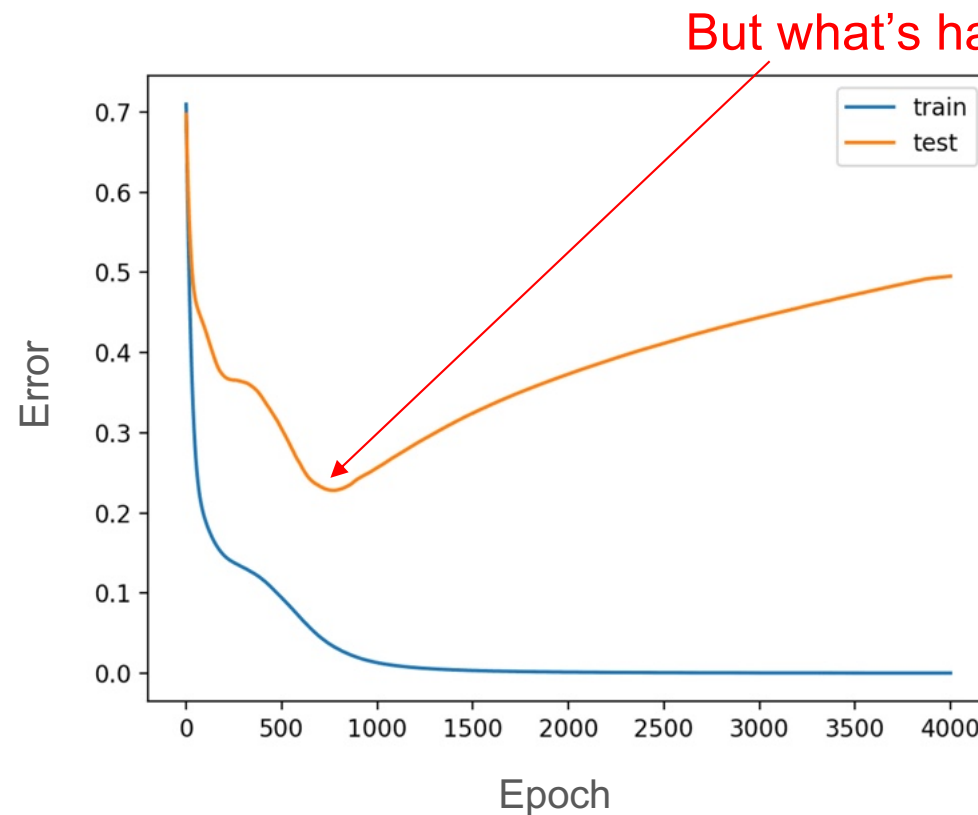
Minimize errors between predictions and true labels



Ball rolling down a hill: start with a random initialization, then stochastic gradient descent by iteratively computing the partial derivatives of the functions

Training ANN through multiple epochs

Error goes down with more training epochs



Training for too long
can lead to
overfitting!

LeNet

Communicated by Dana Ballard

Backpropagation Applied to Handwritten Zip Code Recognition

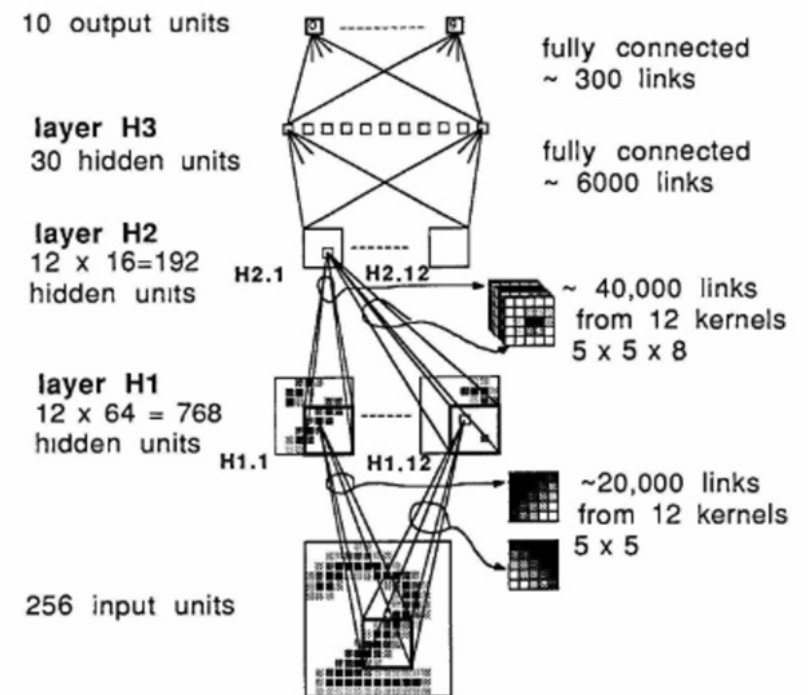
Y. LeCun
B. Boser
J. S. Denker
D. Henderson
R. E. Howard
W. Hubbard
L. D. Jackel

AT&T Bell Laboratories Holmdel, NJ 07733 USA

The ability of learning networks to generalize can be greatly enhanced by providing constraints from the task domain. This paper demonstrates how such constraints can be integrated into a backpropagation network through the architecture of the network. This approach has been successfully applied to the recognition of handwritten zip code digits provided by the U.S. Postal Service. A single network learns the entire recognition operation, going from the normalized image of the character to the final classification.

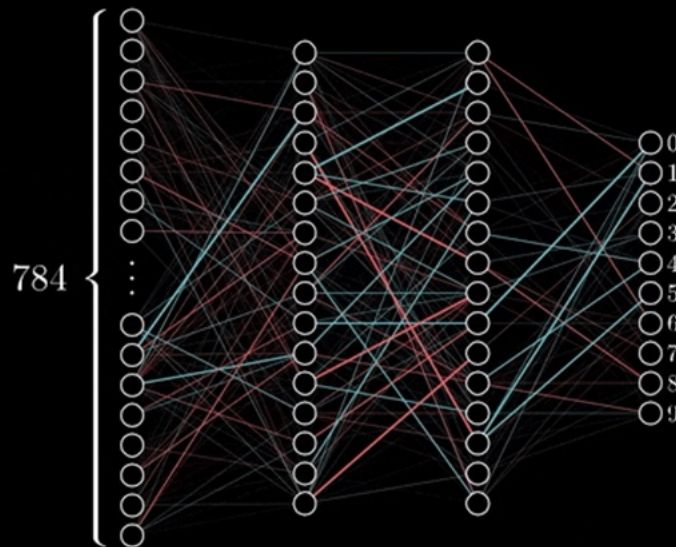
548

LeCun, Boser, Denker, Henderson, Howard, Hubbard, and Jackel

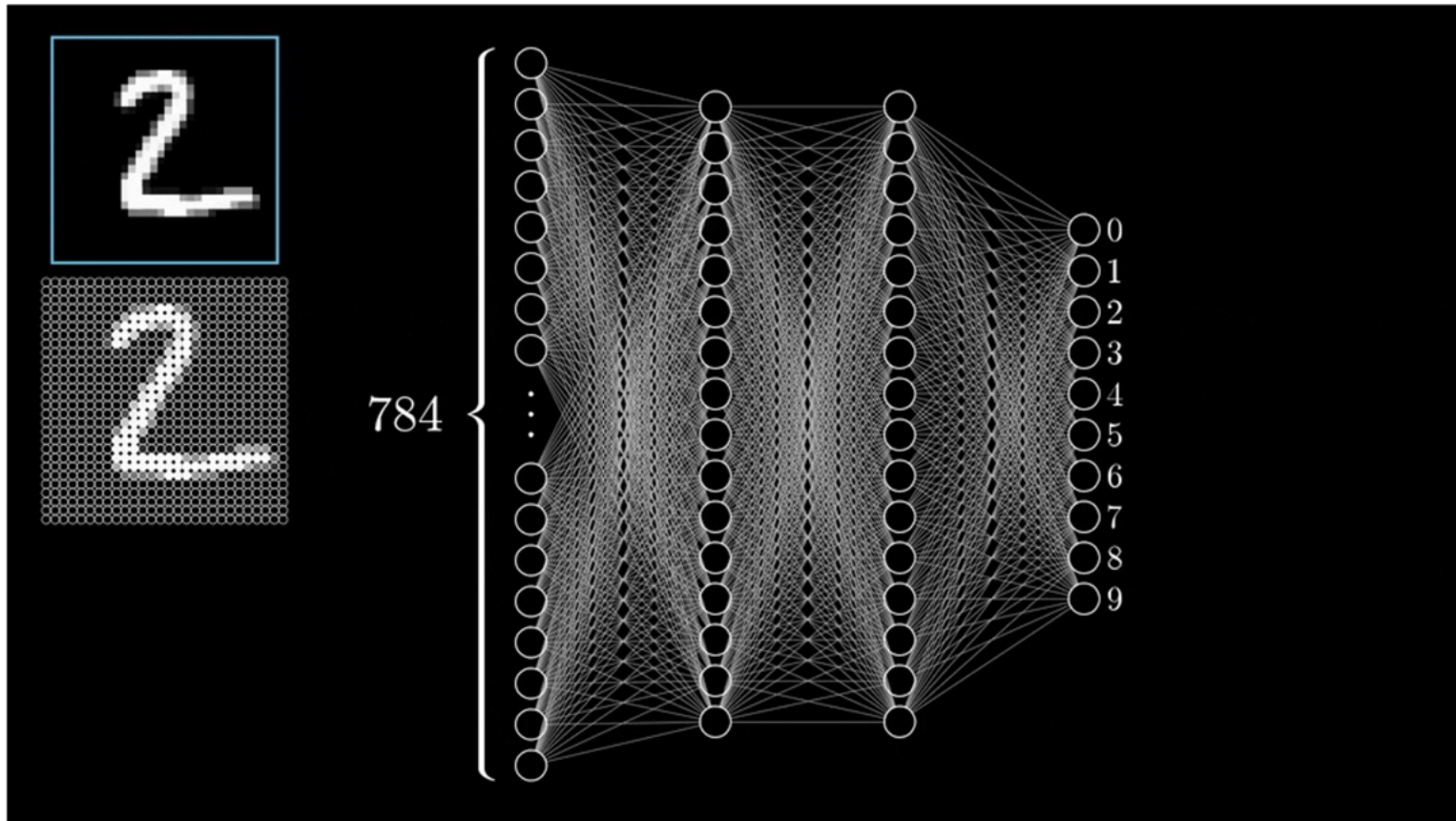


Recap - Training

Training in
progress...



Recap - Evaluation

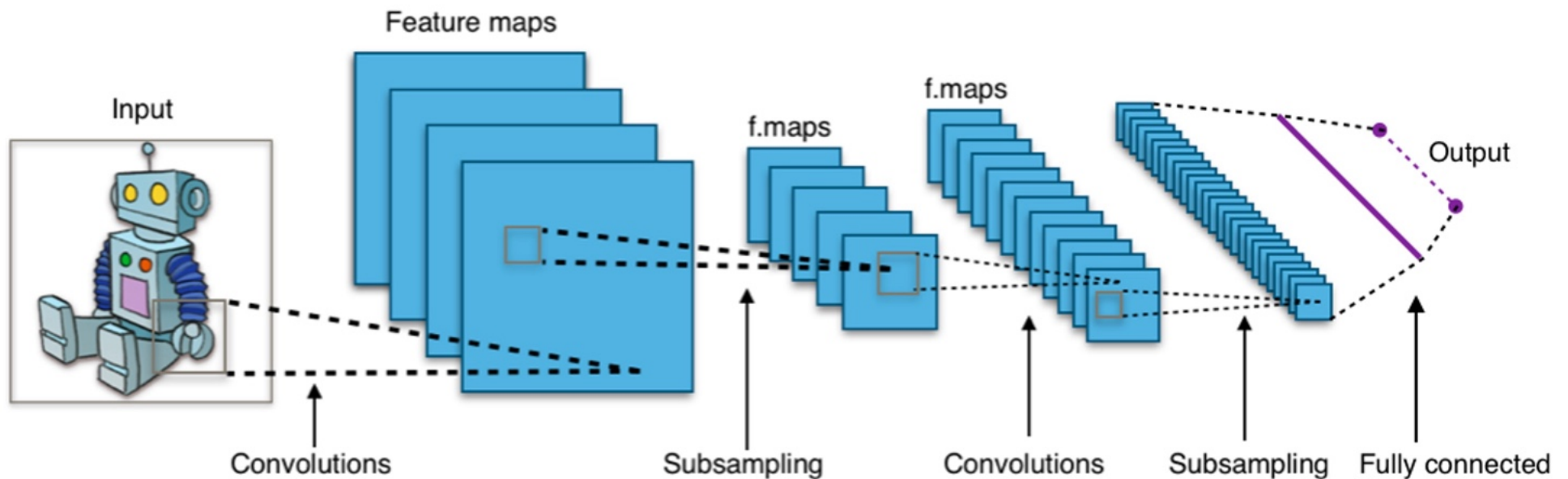


<https://www.youtube.com/watch?v=llg3gGewQ5U>

Outline

1. The Problem
2. pre-AlexNet approaches
3. How do animals & humans approach this?
4. Artificial Neural Networks
5. Convolutional Neural Networks
6. AlexNet
7. Impact

Convolutional Neural Network

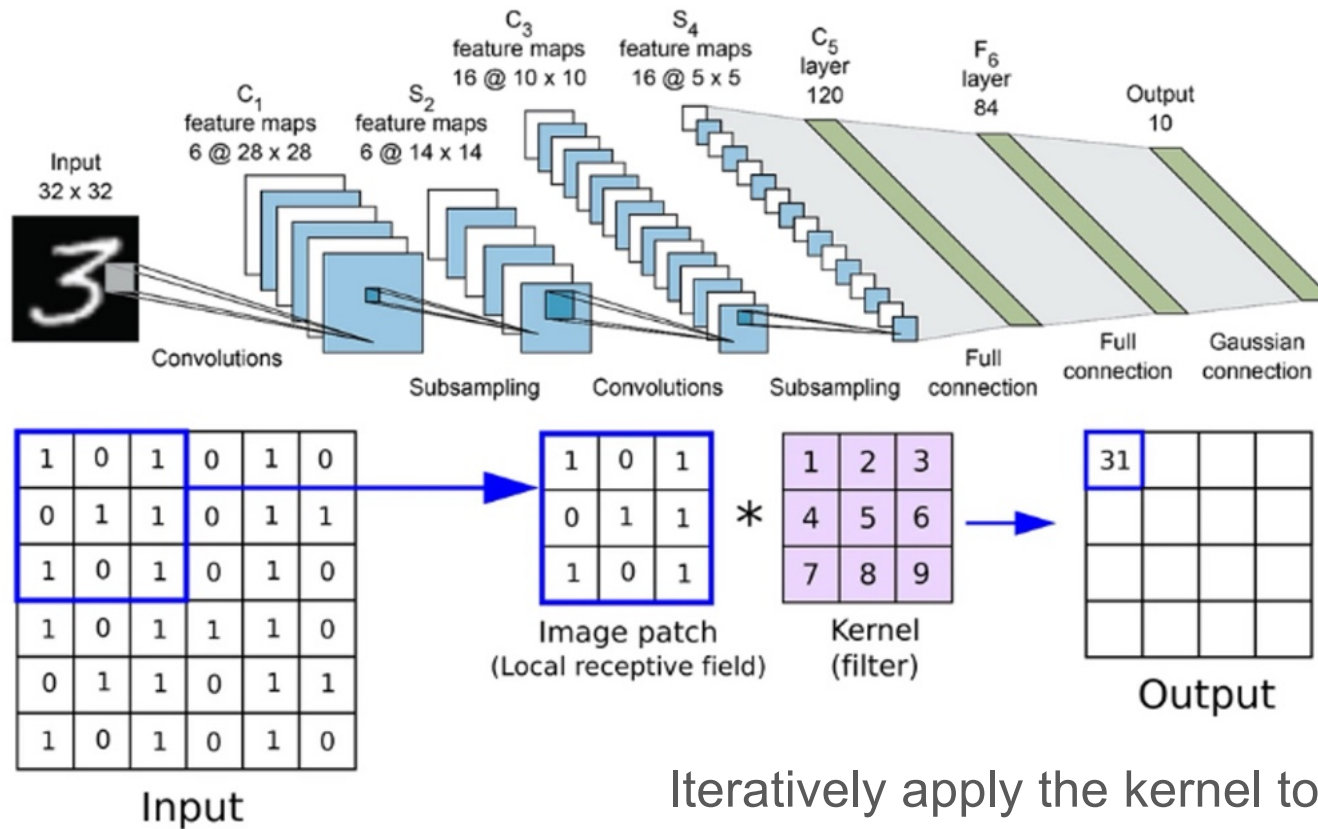


Fully connected neural networks are most expressive but are computationally intensive!

CNNs achieve nearly the same power but with much less compute:

- $O(\text{Nodes} * \text{Depth})$ rather than $O(\text{Nodes} ^ \text{Depth})$

Convolutional Neural Network



- Iteratively apply the kernel to extract features
- Starts from the input image, then creates more abstract derived images

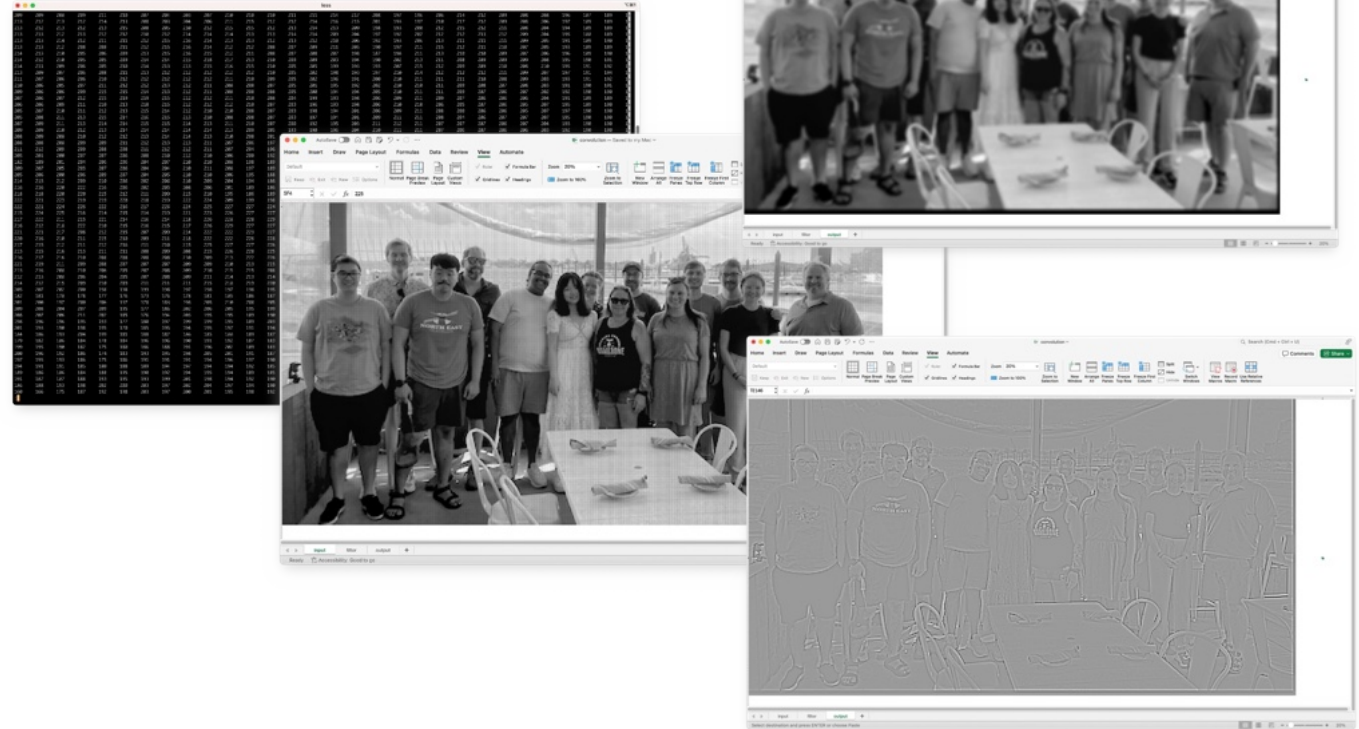
“LeNet” (Lecun et al, 1998)

Kernels

Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 x 3 (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	
Gaussian blur 5 x 5 (approximation)	$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	
Unsharp masking 5 x 5 Based on Gaussian blur with amount as 1 and threshold as 0 (with no image mask)	$\frac{-1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & -476 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	

Kernels

Operation	Kernel ω	Image result g(x,y)
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 x 3 (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	
Gaussian blur 5 x 5 (approximation)	$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	
Unsharp masking 5 x 5 Based on Gaussian blur with amount as 1 and threshold as 0 (with no image mask)	$\frac{-1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & -476 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	



$$= \text{input!A1} * \text{filter!SA\$1} + \text{input!A2} * \text{filter!SA\$2} + \text{input!A3} * \text{filter!SA\$3} + \text{input!A4} * \text{filter!SA\$4} + \text{input!A5} * \text{filter!SA\$5} +$$

$$\text{input!B1} * \text{filter!SB\$1} + \text{input!B2} * \text{filter!SB\$2} + \text{input!B3} * \text{filter!SB\$3} + \text{input!B4} * \text{filter!SB\$4} + \text{input!B5} * \text{filter!SB\$5} +$$

$$\text{input!C1} * \text{filter!SC\$1} + \text{input!C2} * \text{filter!SC\$2} + \text{input!C3} * \text{filter!SC\$3} + \text{input!C4} * \text{filter!SC\$4} + \text{input!C5} * \text{filter!SC\$5} +$$

$$\text{input!D1} * \text{filter!SD\$1} + \text{input!D2} * \text{filter!SD\$2} + \text{input!D3} * \text{filter!SD\$3} + \text{input!D4} * \text{filter!SD\$4} + \text{input!D5} * \text{filter!SD\$5} +$$

$$\text{input!E1} * \text{filter!SE\$1} + \text{input!E2} * \text{filter!SE\$2} + \text{input!E3} * \text{filter!SE\$3} + \text{input!E4} * \text{filter!SE\$4} + \text{input!E5} * \text{filter!SE\$5}$$